1.Создала бд и проинициализировала данными pgbench -i -s 50 -d bench_db

```
vacuuming...
creating primary keys...
done in 21.32 s (drop tables 0.00 s, create tables 0.01 s, client-side generate
15.46 s, vacuum 1.12 s, primary keys 4.72 s).
```

2.Прогоны без скрипта вручную.

Без расширения pgbench -c 10 -j 2 -t 1000 -d bench_db

1 прогон

```
user@MacBook-Air-user ~ % pgbench -c 10 -j 2 -t 1000 -d bench_db
pgbench (18.4)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 4.197 ms
initial connection time = 36.557 ms
tps = 2382.887058 (without initial connection time)
```

- TPS: **2382.88**
- Latency: **4.197 ms**

2 прогон

```
user@MacBook-Air-user ~ % pgbench -c 10 -j 2 -t 1000 -d bench_db
pgbench (18.4)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 3.551 ms
initial connection time = 37.203 ms
tps = 2816.480920 (without initial connection time)
```

- TPS: **2816.48**
- Latency: **3.551 ms**

3 прогон

```
user@MacBook-Air-user ~ % pgbench -c 10 -j 2 -t 1000 -d bench_db
pgbench (18.4)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 3.612 ms
initial connection time = 33.770 ms
tps = 2768.561544 (without initial connection time)
```

- TPS: **2768.56**
- Latency: **3.612 ms**

3.Создала расширение в бд

```
user@MacBook-Air-user ~ % psql -d postgres -c "ALTER SYSTEM SET shared_preload_libraries = 'pg_stat_statements';"
ALTER SYSTEM
user@MacBook-Air-user ~ % pg_ctl -D ~/postgres/data/debug restart
waiting for server to shut down.... done
server stopped
waiting for server to start....2026-10-21 20:06:30.677 MSK [40131] LOG:  starting PostgreSQL 18.4 on x86_64-apple-darwin24.6.0,
compiled by Apple clang version 17.0.0 (clang-1700.0.13.5), 64-bit
2026-10-21 20:06:30.679 MSK [40131] LOG:  listening on IPv6 address "::1", port 5432
2026-10-21 20:06:30.679 MSK [40131] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2026-10-21 20:06:30.681 MSK [40131] LOG:  listening on Unix socket "/tmp/.s.PGSQL.5432"
2026-10-21 20:06:30.700 MSK [40139] LOG:  database system was shut down at 2026-10-21 20:06:30 MSK
2026-10-21 20:06:30.707 MSK [40131] LOG:  database system is ready to accept connections
 done
server started
user@MacBook-Air-user ~ % psql -d bench_db -c "CREATE EXTENSION IF NOT EXISTS pg_stat_statements;"
CREATE EXTENSION
```

Выполнила прогоны с расширением

1 прогон

```
user@MacBook-Air-user ~ % pgbench -c 10 -j 2 -t 1000 -d bench_db
pgbench (18.4)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 3.927 ms
initial connection time = 39.724 ms
tps = 2546.556788 (without initial connection time)
```

- TPS: **2546.56**
- Latency: **3.927 ms**

2 прогон

```
user@MacBook-Air-user ~ % pgbench -c 10 -j 2 -t 1000 -d bench_db
pgbench (18.4)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 3.871 ms
initial connection time = 34.638 ms
tps = 2583.582778 (without initial connection time)
```

- TPS: **2583.58**
- Latency: **3.871 ms**

3 прогон

```
user@MacBook-Air-user ~ % pgbench -c 10 -j 2 -t 1000 -d bench_db
pgbench (18.4)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 4.024 ms
initial connection time = 38.874 ms
tps = 2485.232747 (without initial connection time)
```

- TPS: **2485.23**
- Latency: **4.024 ms**
  Средние значения
- Без расширения: TPS = **2655.97**, Latency = **3.786 ms.**
- С расширением: TPS = **2538.46**, Latency = **3.941 ms.**
  Вывод: pg_stat_statements немного снижает производительность (примерно на 4.4% по TPS), потому что тратит ресурсы на сбор статистики.

4.Скачала исходники текущего master (PostgreSQL 20devel) через Git, собрала их в отдельную папку, инициализировала новый кластер. Заполнила базу bench_db данными (pgbench -i -s 50). Выполнила 3 прогона.

1 прогон

```
user@MacBook-Air-user master % ~/postgres/install/master/bin/pgbench
-c 10 -j 2 -t 1000 -p 5434 bench_db
pgbench (20devel)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 3.692 ms
initial connection time = 20.948 ms
tps = 2708.345577 (without initial connection time)
```

- TPS = **2708.34**
- Latency = **3.692 ms**

2 прогон

```
user@MacBook-Air-user master % ~/postgres/install/master/bin/pgbench
-c 10 -j 2 -t 1000 -p 5434 bench_db
pgbench (20devel)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 2.650 ms
initial connection time = 34.571 ms
tps = 3773.250297 (without initial connection time)
```

- TPS = **3773.25**
- Latency = **2.650 ms**

3 прогон

```
user@MacBook-Air-user master % ~/postgres/install/master/bin/pgbench
-c 10 -j 2 -t 1000 -p 5434 bench_db
pgbench (20devel)
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 50
query mode: simple
number of clients: 10
number of threads: 2
maximum number of tries: 1
number of transactions per client: 1000
number of transactions actually processed: 10000/10000
number of failed transactions: 0 (0.000%)
latency average = 2.406 ms
initial connection time = 31.370 ms
tps = 4156.574849 (without initial connection time)
```

- TPS = **4156.57**
- Latency = **2.406 ms**
  Средние значения
- TPS = **3546.05**, Latency = **2.916 ms**
  Вывод: Текущая версия master имеет прирост производительности около 33% по TPS и снижение latency в среднем на 0.8–0.9 мс по сравнению с релизной.

5.Подготовила скрипт для сравнения прогон pgbench дважды (с включенным и выключенным расширением)

Скрипт run_experiment.sh (https://github.com/cherkaasova/PostgresPro/blob/master/reports/run_experiment.sh):

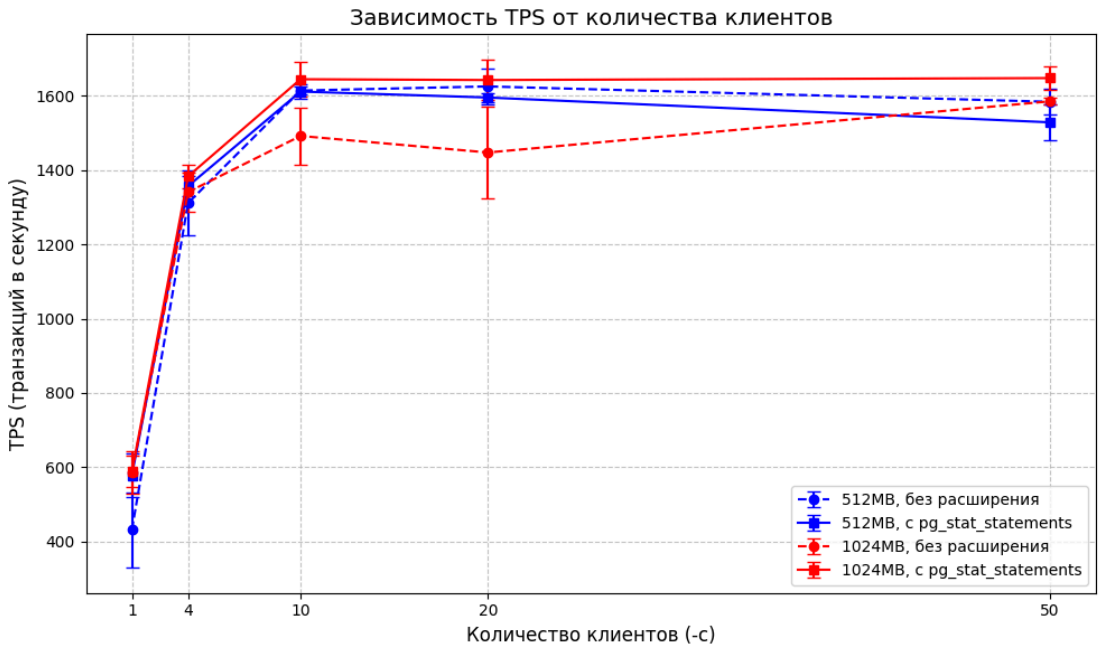
1. Перебирает 4 комбинации: `512MB ext=on`, `512MB ext=off`, `1024MB ext=on`, `1024MB ext=off`.
2. Для каждой комбинации:

- Меняет shared_buffers и shared_preload_libraries через ALTER SYSTEM
- Перезапускает сервер (pg_ctl restart).

3. Для каждой комбинации × 5 клиентов (1, 4, 10, 20, 50) × 3 повтора:

- Запускает `pgbench -c <clients> -j 2 -T 30`.
- Извлекает `tps` и latency_avg из вывода.
- Пишет строку в CSV ext_compare.csv.

Обработала результаты из CSV с помощью Jupyter Notebook (https://github.com/cherkaasova/PostgresPro/blob/master/reports/graphs.ipynb)



Вывод: shared_buffers: 1024MB даёт прирост TPS по сравнению с 512MB (заметнее на малых нагрузках), так как база полностью помещается в кеш. TPS стабильно выше с выключенным pg_stat_statements. После 10 клиентов TPS почти не растёт, так как процессор уже загружен полностью. Конфигурация 1024MB, off показала максимальный TPS.